

Algorithm Map — theory ↔ implementation

Maps each hand-waved step in `QuinticMethods_v1.md` to the concrete algorithm the simulator actually runs, plus the pitfalls discovered and verified during the build (2026-08-15, on this machine).

Stage map

Stage	Module	Concrete algorithm	Notes / pitfalls
S0 input	'input.py'	exact rational normalization; square-free split via 'gcd(f, f')' ('Poly.sqf_list')	rejects degree $\neq 5$ and non-rational coefficients
S1 normal form	'normalform.py'	exact substitution 'x = y - a ₄ /5'; palindromic detection + reduction	Bring-Jerrard skipped (coefficient blow-up, plan risk #3); see palindromic correction below
S2 factor	'factor.py'	Rational Root Theorem by exact divisor enumeration, then Zassenhaus (SymPy 'factor_list': GF(p) factor + Hensel lift + Mignotte-bound reconstruction)	No LLL in SymPy's path (verified: zero LLL references in 'sympy/polys/factortools.py'). LLL-based reconstruction is PARI's approach, reachable only via the Sage cross-check
S3 Galois group	'galois.py'	SymPy 'galois_group(f, by_name=True)' = '_galois_group_degree_5' — Cohen Alg 6.3.9 — sextic resolvent R ₅₁ (coefficient lookup) + discriminant squareness + quadratic resolvent R ₂ , Tschirnhaus retries	No LLL, no Stauduhar at degree 5: identification degenerates to hybrid table (order + A ₅ -membership). Stauduhar is PARI/Sage's higher-degree machinery. SymPy calls F ₂₀ "M20". Raises on reducible input — S2 must run first
S3 cross-check	'sage_bridge.py'	Sage-in-Docker 'f.galois_group()' (Cohen resolvent + table lookup internally)	verified Docker protocol below; transitive group number N = 1...5 → C5/D5/F20/A5/S5

S4a radicals	'radicals.py'	SymPy 'roots_quintic': depress → sextic F₂₀ resolvent → integer root θ → La-grange resolvents in ζ_5 → fifth roots (the Brioschi/Kro-necker/Dedekind resolvent family)	F_{20} resolvent + θ exposed in the trace; returns '{}' for A_5/S_5
S4b special	'special.py'	'CRootOf' exact algebraic numbers (Vincent inter- val isolation via 'real_roots')	Bring radical = notation only (formula ' $x =$ $p^{1/4} \cdot \text{Br}(-q \cdot p^{(-5/4)})$ ')) verified numer- ically); Klein = documented stub (TODO future)
S5 numerics	'numeric.py'	mpmath 'poly- roots' (Aberth , 50 digits) + NumPy companion-matrix 'eigvals' + Durand- Kerner (roots-of- unity seeding, con- vergence history)	coefficient- ordering trap : mpmath wants de- scending, NumPy 'polycompanion' wants ascending — one adapter. Aberth does not converge on multi- ple roots → NumPy fallback (machine precision)
S6 gate	'verify.py'	40-digit residual '	f(r)
S7 report	'report.py', 'pipeline.py', 'cli.py'	'Report' dataclass → Markdown + JSON (stable schema), 'Step- Trace' records with wall times	CLI: 'python -m quintic_sim "..." [-verbose] [-sage] [-json out]'

Verified pitfalls (all hit and fixed during the build)

1. **Palindromic correction (document discrepancy, V10).** A palindromic monic quintic $x^5 + a x^4 + b x^3 + b x^2 + a x + 1$ always has $x = -1$ as a root. The remaining palindromic quartic reduces to a **quadratic** in $t = x + 1/x$: $t^2 + (a-1) \cdot t + (b-a-1) = 0$, then two quadratics $x^2 - t x + 1$. QuinticMethods_v1.md §5's "cubic in $x + 1/x$ " is a loose textbook statement; the simulator implements the correct version and prints the correction in the report.
2. **NumPy coefficient ordering (V6).** `np.polynomial.polynomial.polycompanion` wants **ascending** coefficients; `mpmath.polyroots` wants **descending**. Mixing them up silently solves the reciprocal polynomial.
3. **cy pari2 does not build on this Windows host** (no PARI C library) — PARI access goes through the Sage container only, and only as an optional cross-check.
4. **Sage image entryptoint quirk (V7).** The `sagemath/sagemath` image's default entry-point swallows file-argument invocations. The verified working protocol is:

```
docker run --rm --entrypoint /bin/bash sagemath/sagemath:latest -c \
'sage << "SAGEEOF"
<sage script that prints JSON>
SAGEEOF'
```

5. **Aberth on multiple roots.** `mpmath.polyroots` raises `NoConvergence` (even at `maxsteps=2000`) on polynomials with repeated roots (e.g. the palindromic test vector, double root at $x=1$). The NumPy companion matrix handles them; the gate's cross-match tolerance adapts ($1e-6$ instead of $1e-25$) when the numeric reference is the machine-precision fallback.
6. **SymPy `galois_group` return shape.** `by_name=True` returns `(S5TransitiveSubgroups.<C5|D5|M20|A5|S5>, ↪in_A5_flag)` — an *enum*, so order/solvability come from the classification table, and the `in_A5` flag is cross-checked against the discriminant invariant.
7. **float64 degradation in tests.** Comparing 50-digit `mpmath` roots through `complex()` loses precision ($\sim 1e-16$); tests compare via `mpmath.mpf/mpc` or use tolerances that account for the conversion.

Invariants tested

- `disc(f)` is a square in $\mathbb{Q}$ **iff** $G \subseteq A_5$ — checked on the 5 group vectors **and** 200 random irreducible quintics (property test).
- `mpmath` Aberth vs NumPy companion agree to $1e-6$ on 50 random quintics.
- Factorization product reconstructs the input exactly.
- Palindromic reduction's 5 closed-form roots match Aberth numerically.
- Bring-radical formula $p^{1/4} \cdot \text{Br}(-q \cdot p^{-5/4})$ reproduces the real root of $x^5 + 5x - 12$ to $1e-30$.
- Sage and SymPy agree on all 5 group vectors (real Docker run).